Week 4 L4.1:
Time Series Machine Learning

COMP3222: Machine Learning Technologies



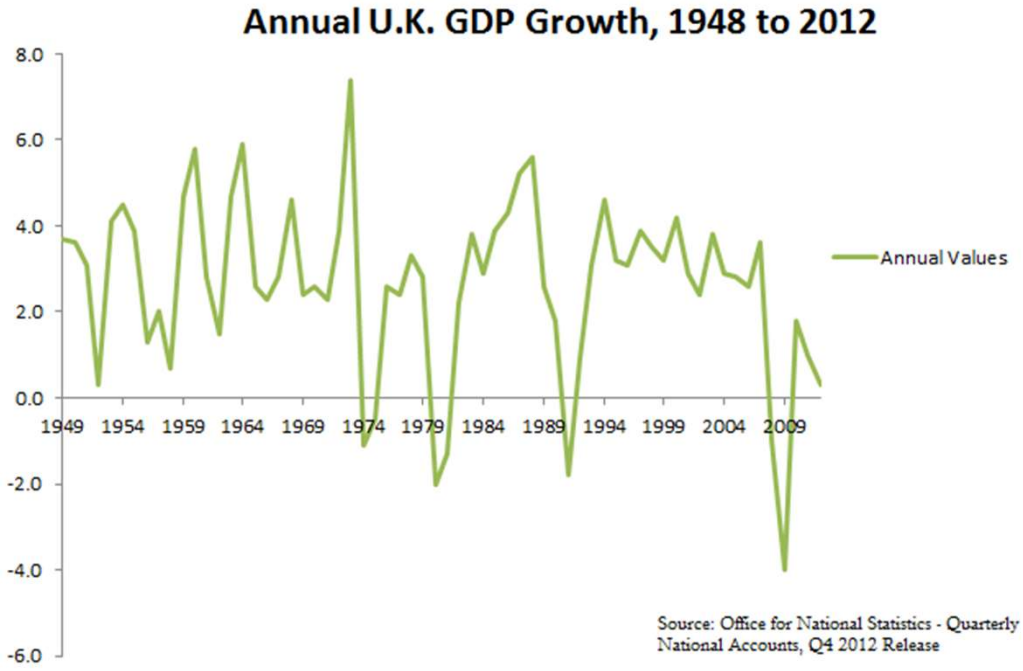
University of Southampton

COMP3222 Lecture Structure

1. Time series data: cases, channels and timepoints
2. Learning tasks and choosing the target
3. A classification example with aeon
4. Distances, alignment and nearest neighbours

What is time series data?

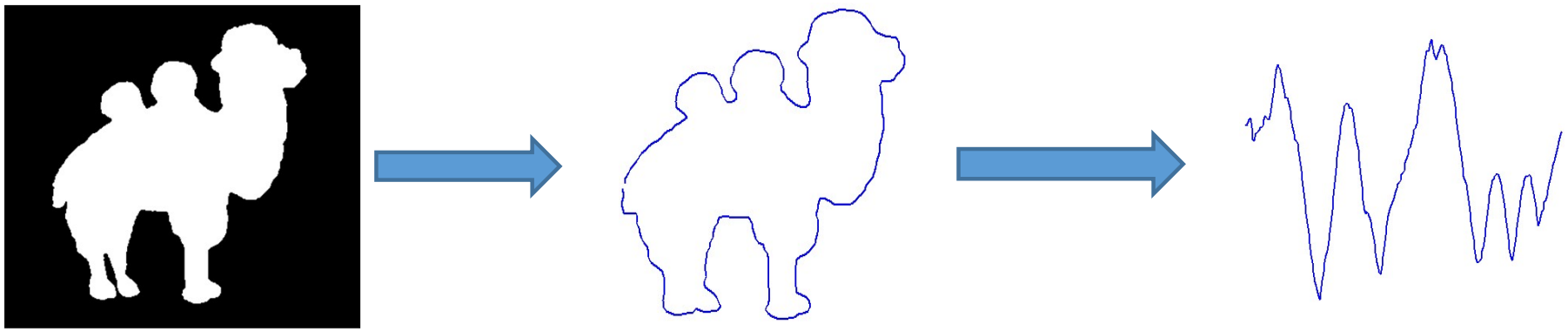
- A time series is a sequence of observations recorded over time. Here we focus on real-valued data.
- If an observation is a scalar, we call it a univariate time series
- If several variables are recorded together, we call it a multivariate time series



Time(?) Series

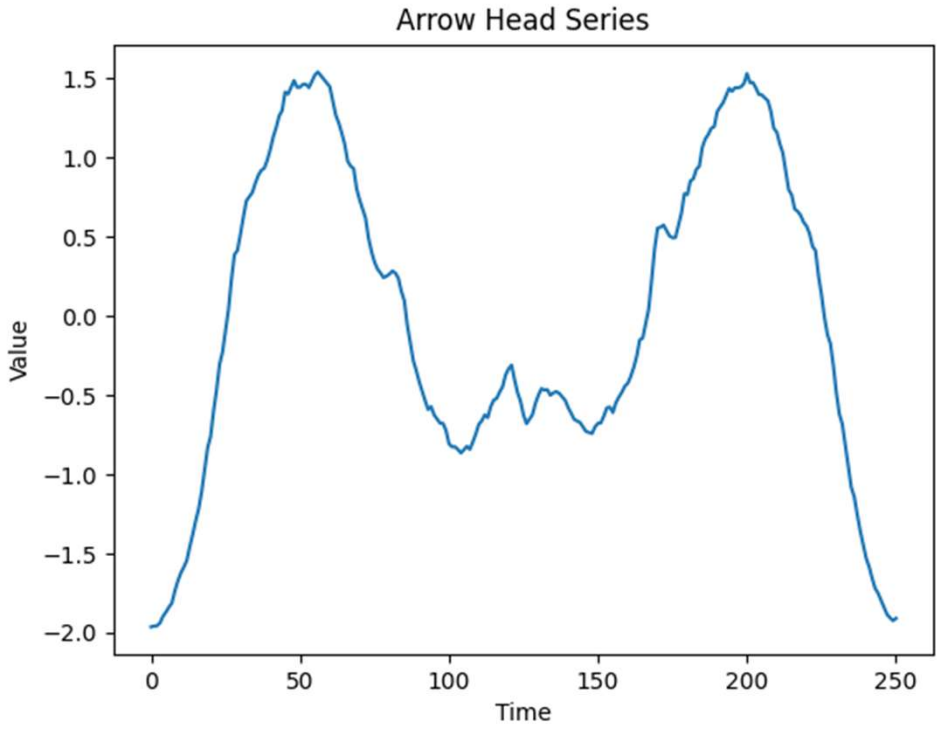
The ordering can represent something other than time, such as position around an outline or wavelength in a spectrum.

For example, an image outline:



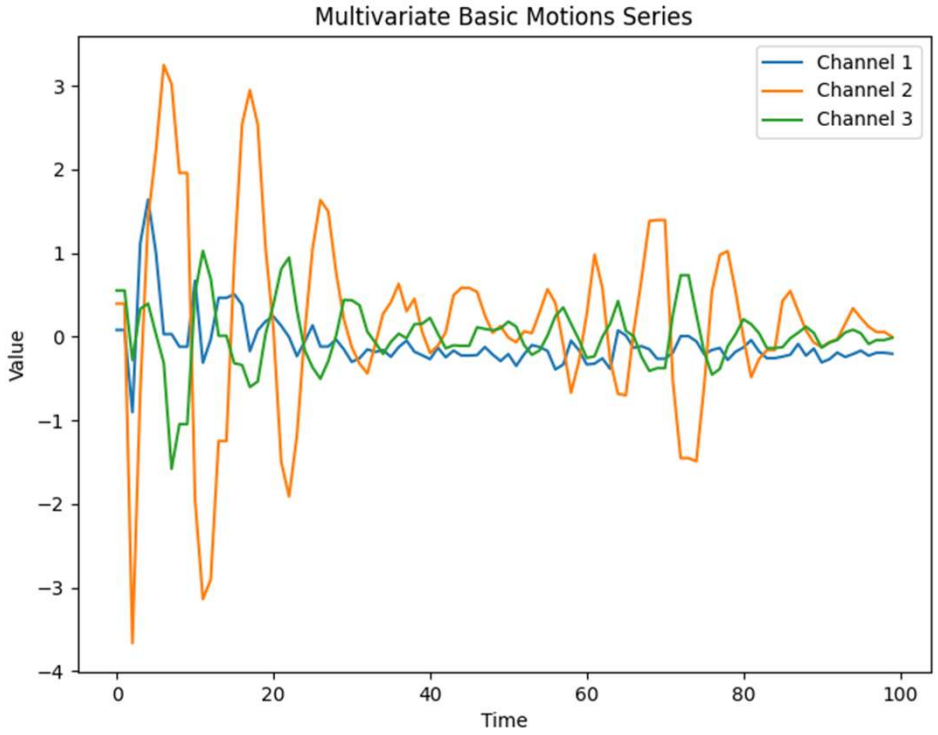
A Single Time Series

- Ordered data, predominantly recorded over time
- One channel with m ordered measurements
- A value is recorded for each of the *m* timepoints
- Useful patterns may cover the whole series or only a short subsequence



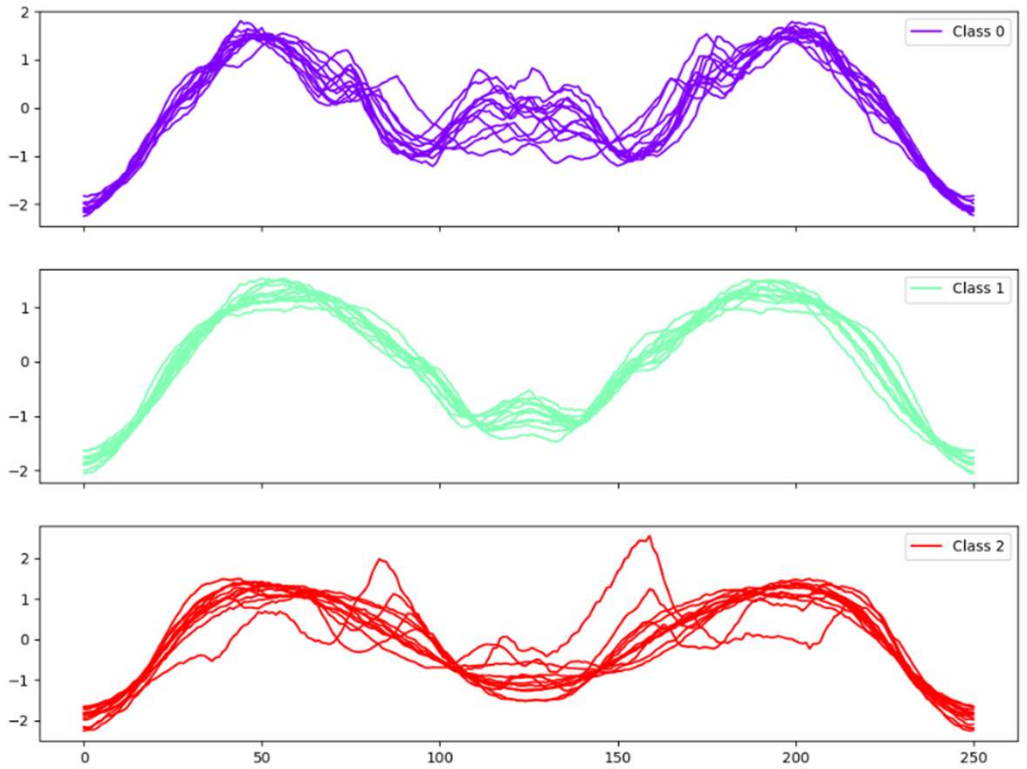
Multivariate Time Series

- Multiple related time series generated concurrently
- For example, acceleration along the X, Y and Z axes of one sensor
- Two-dimensional, with the number of channels (*d*) and series length (*m*)
- Each channel measures a different variable for the same case



A Collection of Time Series

- Multiple recordings form a collection of time series
- Three-dimensional, with the number of cases (*n*), number of channels (*d*) and series length (*m*)
- For equal-length data, store cases, channels and timepoints in a 3D array









Time Series Data in NumPy

Array or expression	Meaning
X.shape = (100, 3, 200)	100 cases, 3 channels, 200 timepoints
X[0]	One multivariate case: shape (3, 200)
X[0, 1]	One channel of that case: shape (200,)
y.shape = (100,)	One target value per case

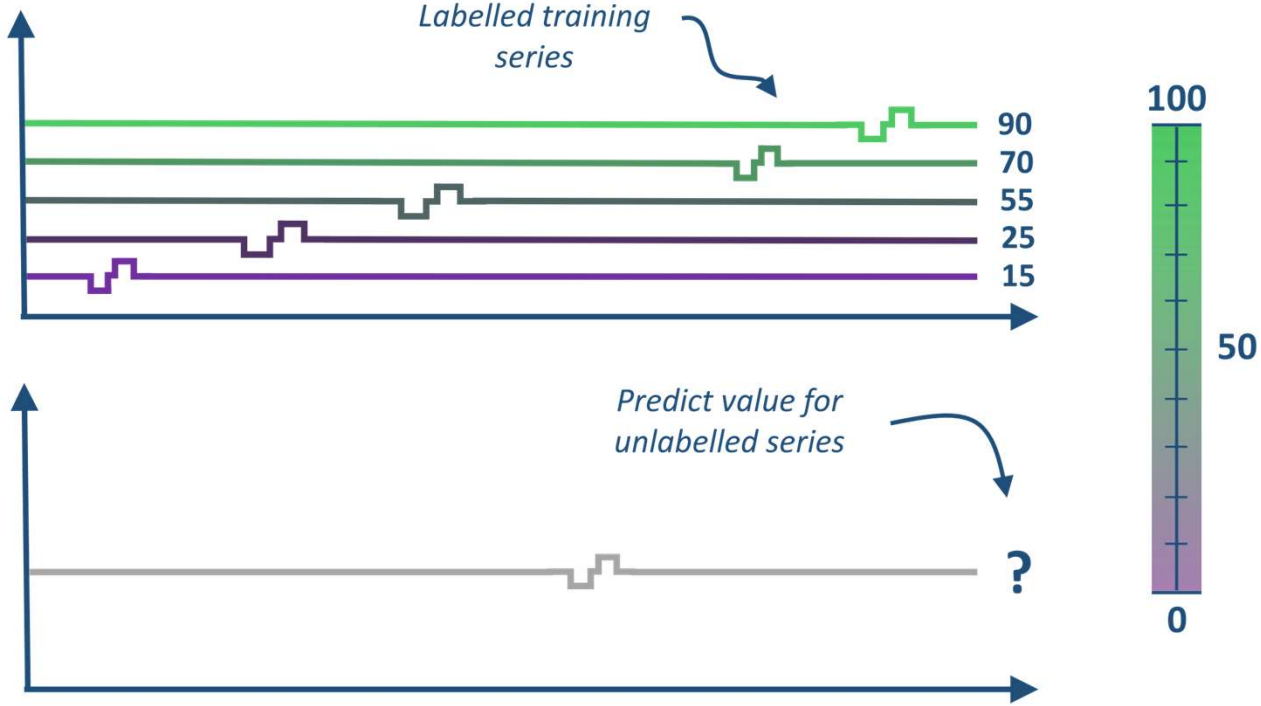
Univariate collections still have a channel axis: (n_cases, 1, n_timepoints).

Time Series Learning Tasks

Task	What do we predict or find?
Classification	A class for each series
Regression	A numeric target for each series
Clustering	Groups of similar series
Forecasting	Future values of a series
Segmentation / anomaly detection	Changes of state / unusual observations
Similarity search	Query a series for closest subseries

Time Series (Extrinsic) Regression (TSR/TSER)

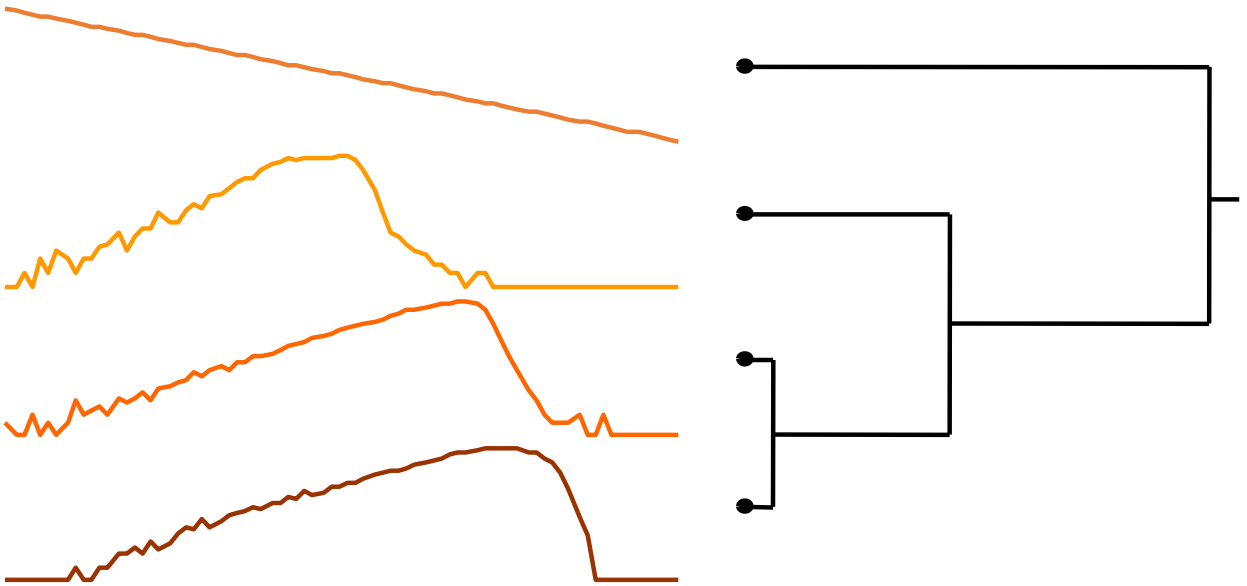
- Rather than a set of class values, we want to predict a continuous target variable
- Algorithms are similar to classification, many of the feature extraction methods remain the same
- Example: predict the current humidity from a temperature recording. Forecasting instead predicts future temperature values.



Time Series Clustering (TSCL)

- Group time series that are similar into the same cluster
- Keep dissimilar series in different clusters

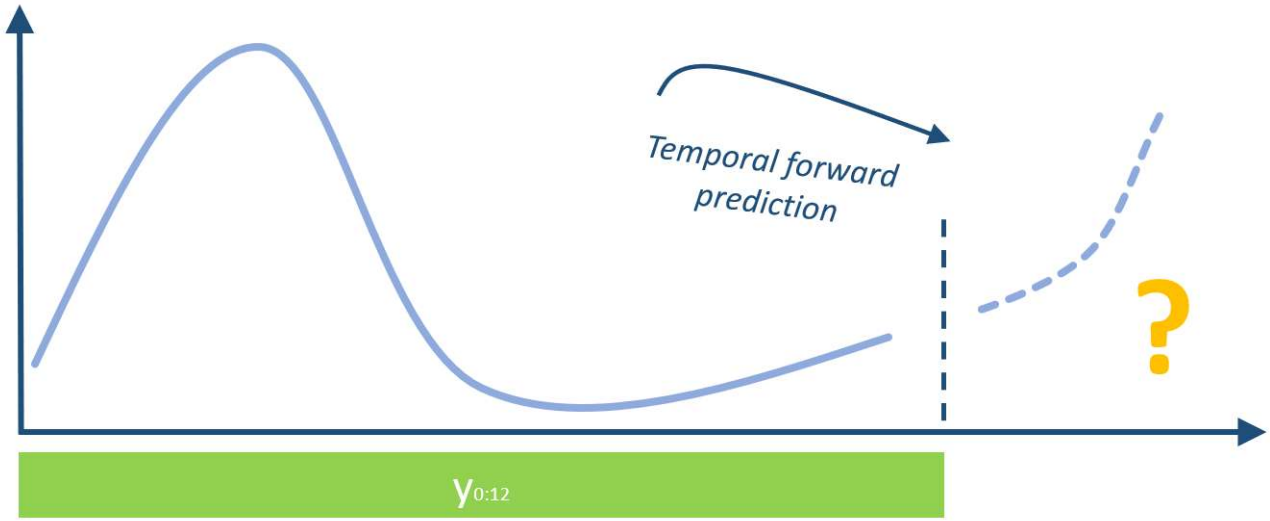
What does it mean for two time series to be similar?



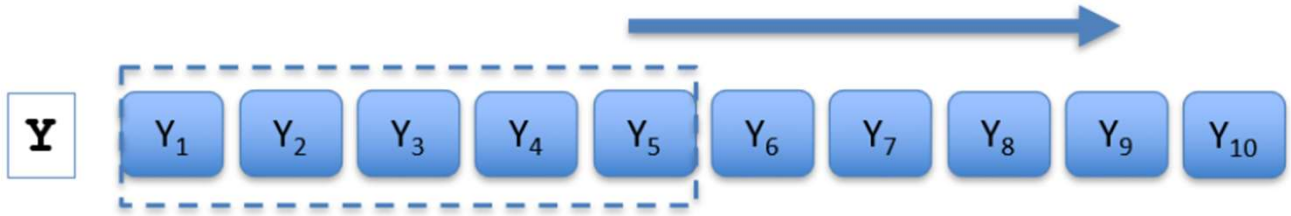
Time Series Forecasting

- Predict future values of a time series based on past observations

- Useful concepts:
  - Trend
  - Seasonality
  - Autocorrelation
  - Exogenous Variables

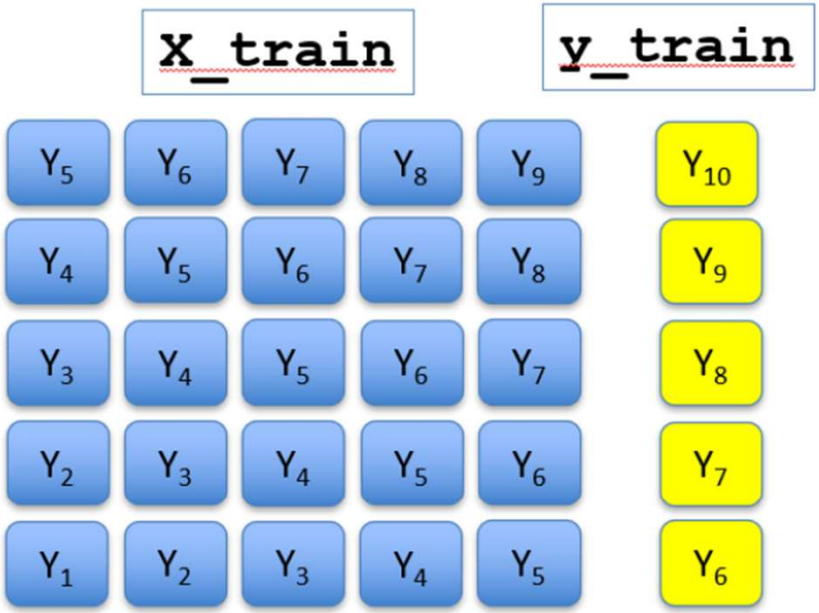


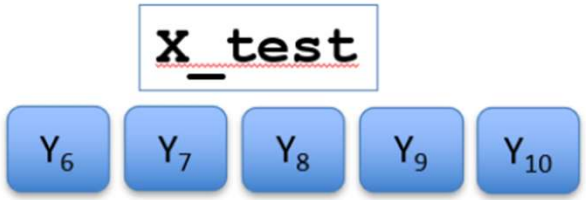
Often converted to a regression problem by using a sliding window



Any regressor will work

```
fp = FreshPRINCE()
fp.fit(X_train,y_train)
```





Time Series Anomaly Detection

- A potentially undesired rare point or rare sequence of points of a given length.
- Shown: NYC taxi cab usage
- Unexpected rises and dips in passengers due to clock changes and weather
- Expected dip during holidays ie.g. Christmas

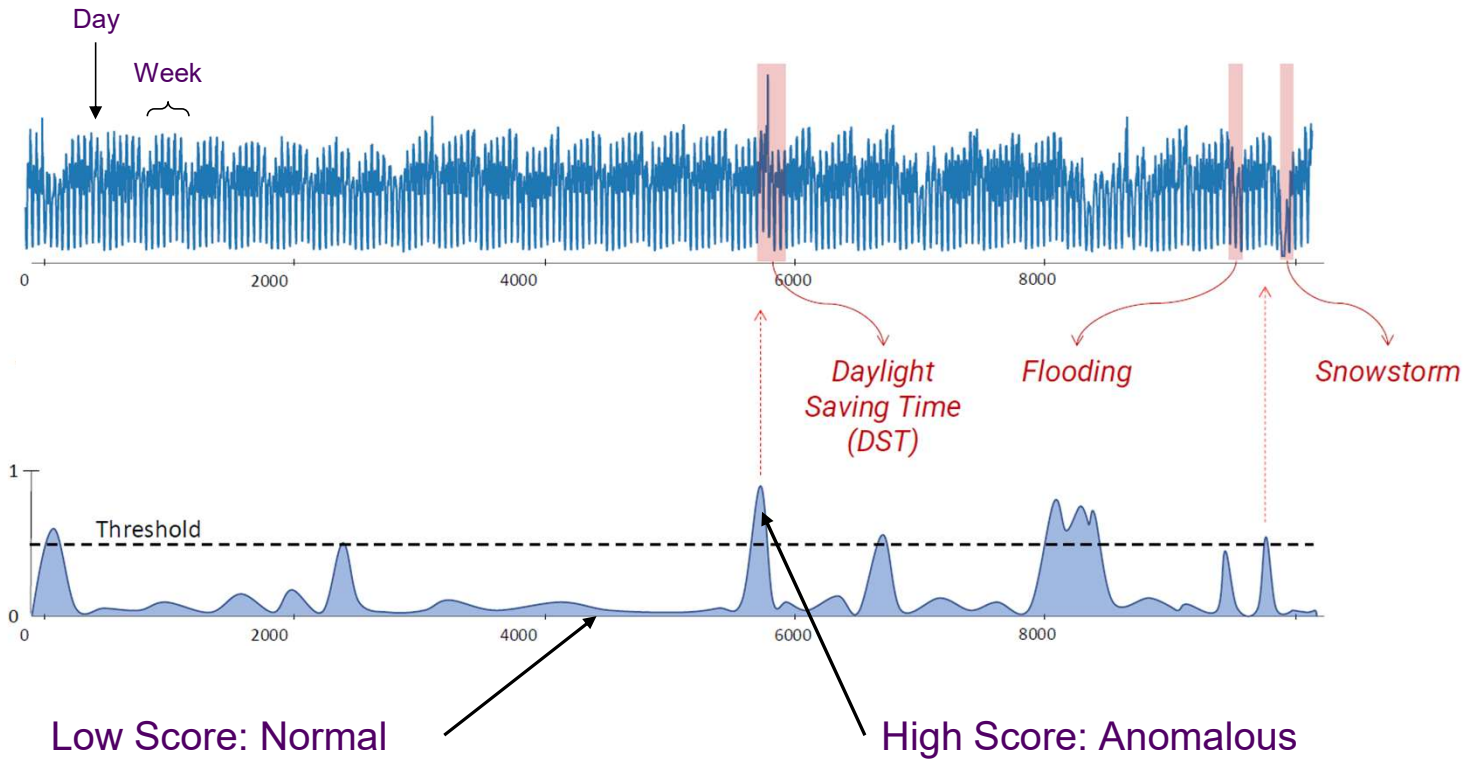
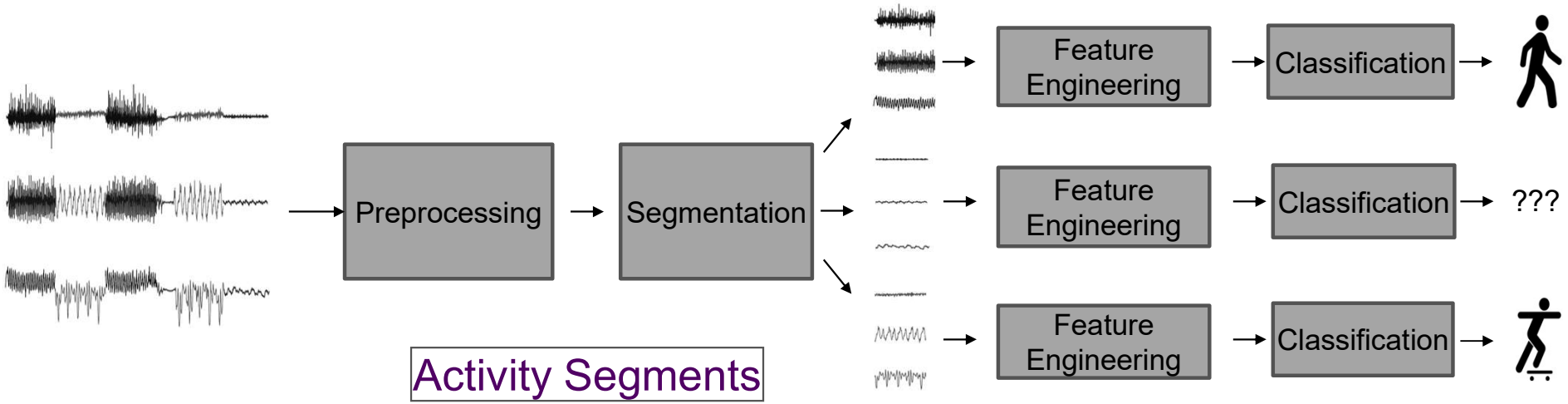


Image from Boniol et al. Time-Series Anomaly Detection: Overview and New Trends. VLDB 2024

Segmentation Example: HAR

A continuous sensor stream must be divided into cases before a classifier can be trained.

Choose windows or activity segments, then assign a label to each case.



Running Example: GunPoint

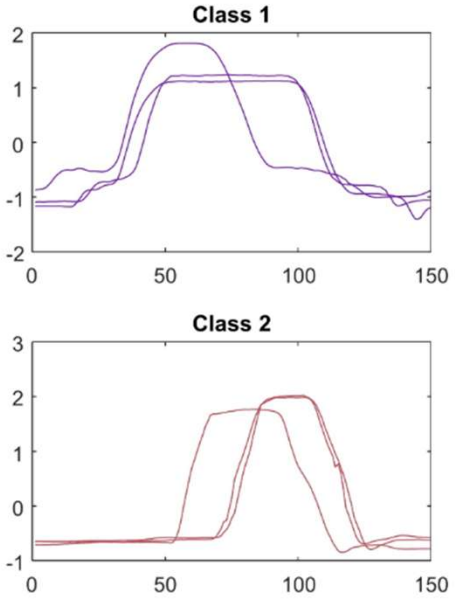
Two classes of arm movement, represented by one measured trajectory

One case is a complete movement, with 150 timepoints

Supplied split: 50 training cases and 150 test cases.

Classification task 1: can we recognise the movement from its shape?

Alternativee classification task: can we recognise the person?





Left) GunPoint recording of 2003, *right*) GunPoint recording of 2018. *Top*) Ann Ratanamahatana, *bottom*) Eamonn Keogh. The female and male actors are the same individuals recorded fifteen years apart.

Loading a Collection with aeon

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from aeon.datasets import load_gunpoint

X_train, y_train = load_gunpoint(split="train")
X_test, y_test = load_gunpoint(split="test")

print(X_train.shape, y_train.shape)
print(X_test.shape, y_test.shape)

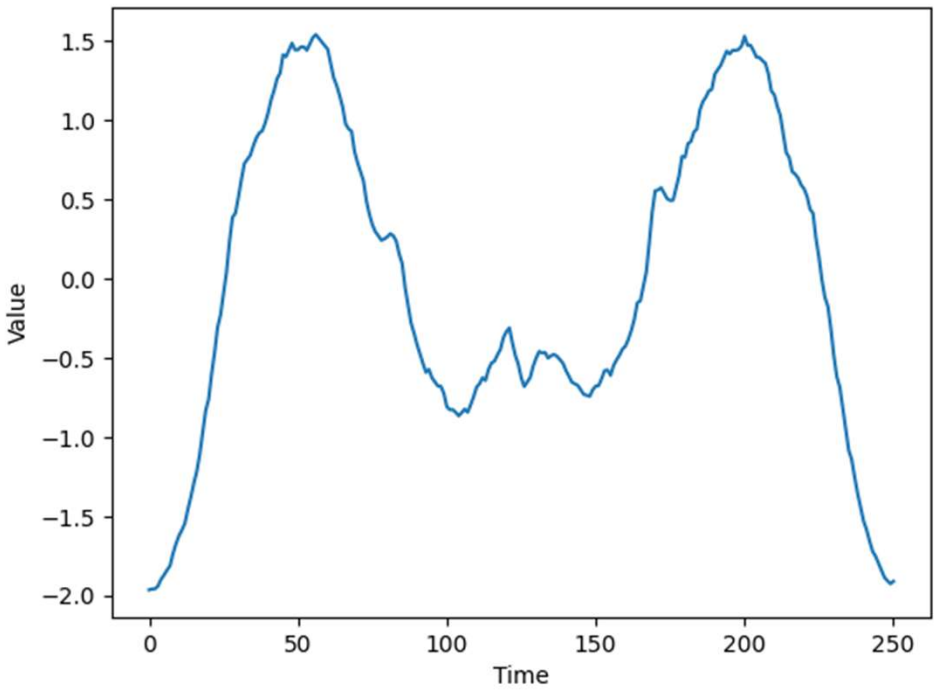
rf = RandomForestClassifier()
rf.fit(X_train[:, 0, :], y_train)
pred = rf.predict(X_test[:, 0, :])
print(accuracy_score(y_test, pred))
```

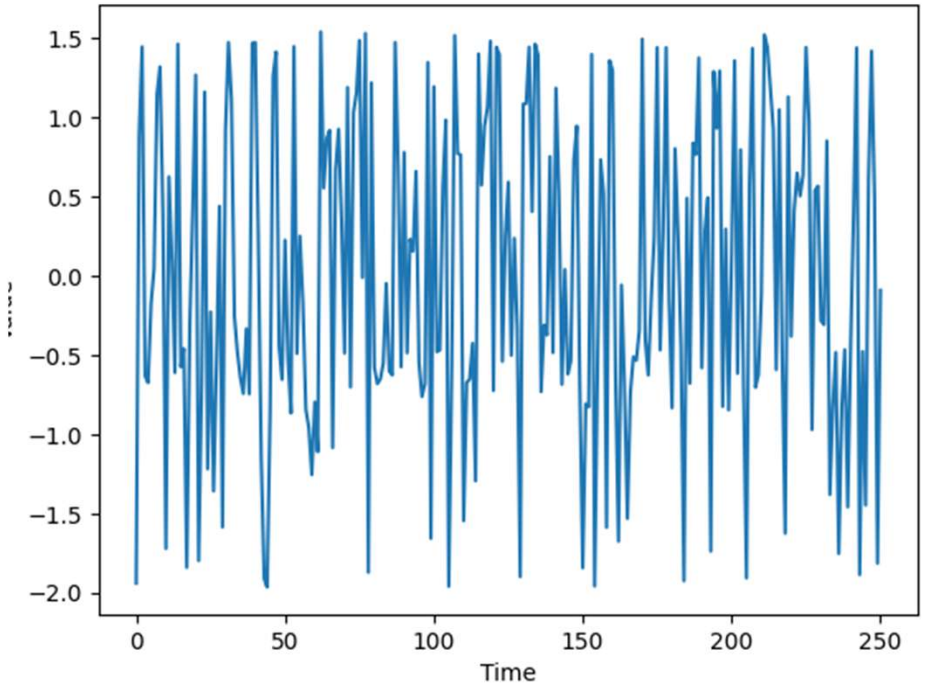
Training: (50, 1, 150), (50,)
Test: (150, 1, 150), (150,)

Each timepoint becomes an attribute. Removing the channel axis gives a 2D array.

The Ordered Part is (usually) Important

Tabular classifiers do not care about the order of the features. Permutation of tabular columns preserves their information but shuffling timepoints removes shape based features.

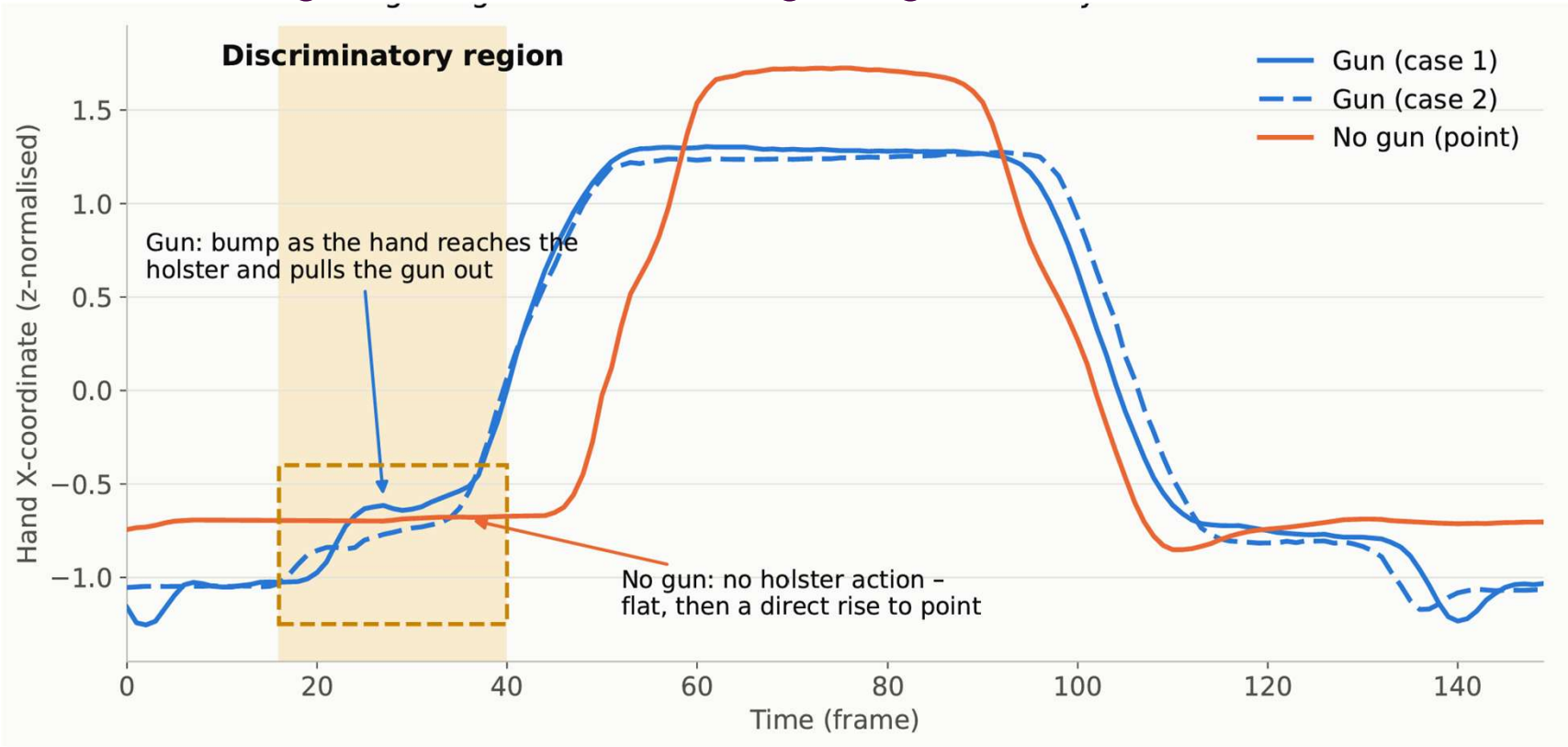




What Sort of Temporal Information Can we Find?

- Suppose we can distinguish between Gun/NoGun using the fact there is a slight movement for the gun case when drawing the gun from a holster

This happens at approximately the same time, but will it be perfectly aligned?



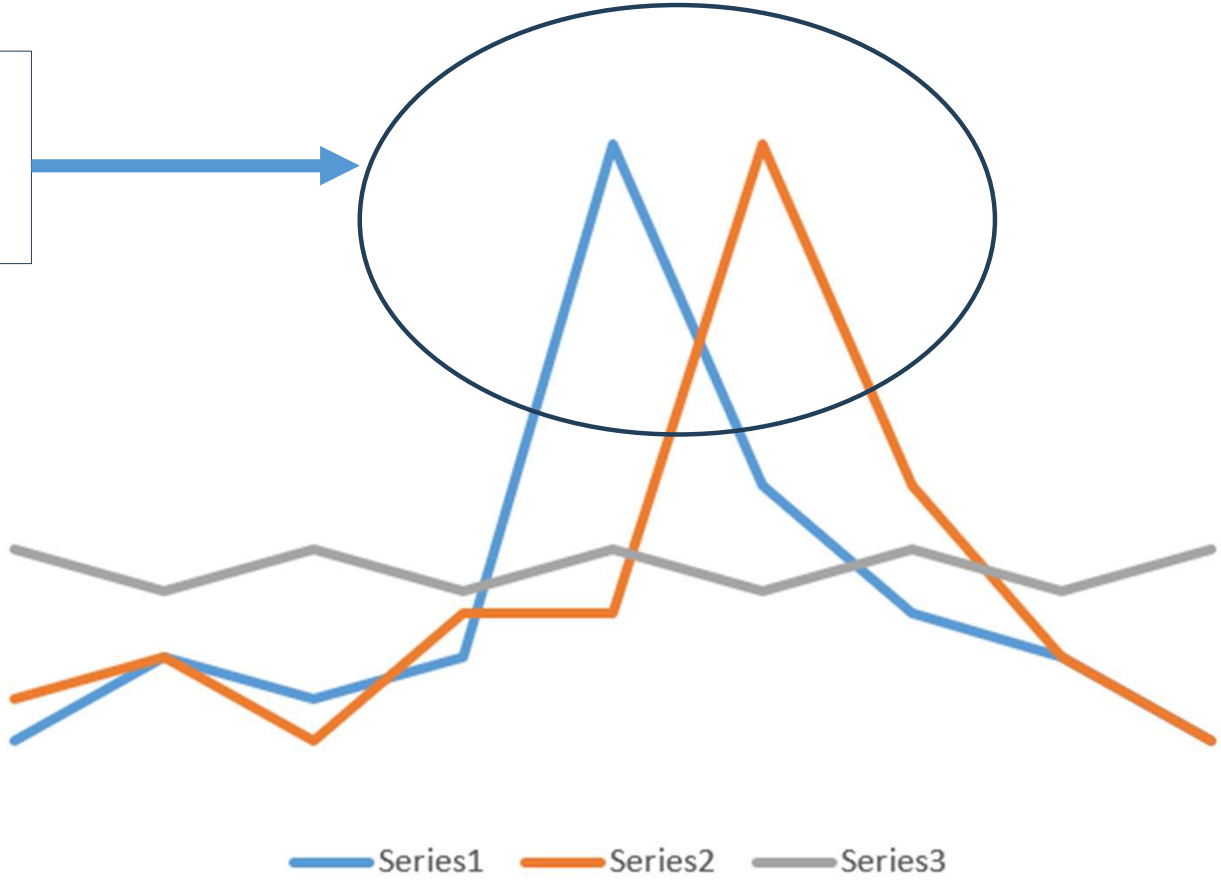
The issue with Euclidean Distance

Slight offset for gun case when drawing the gun from the holster

Is series 1 more like series 2 or series 3?

ed_dist(s1, s2) = 14.03

ed_dist(s1, s3) = 12.58



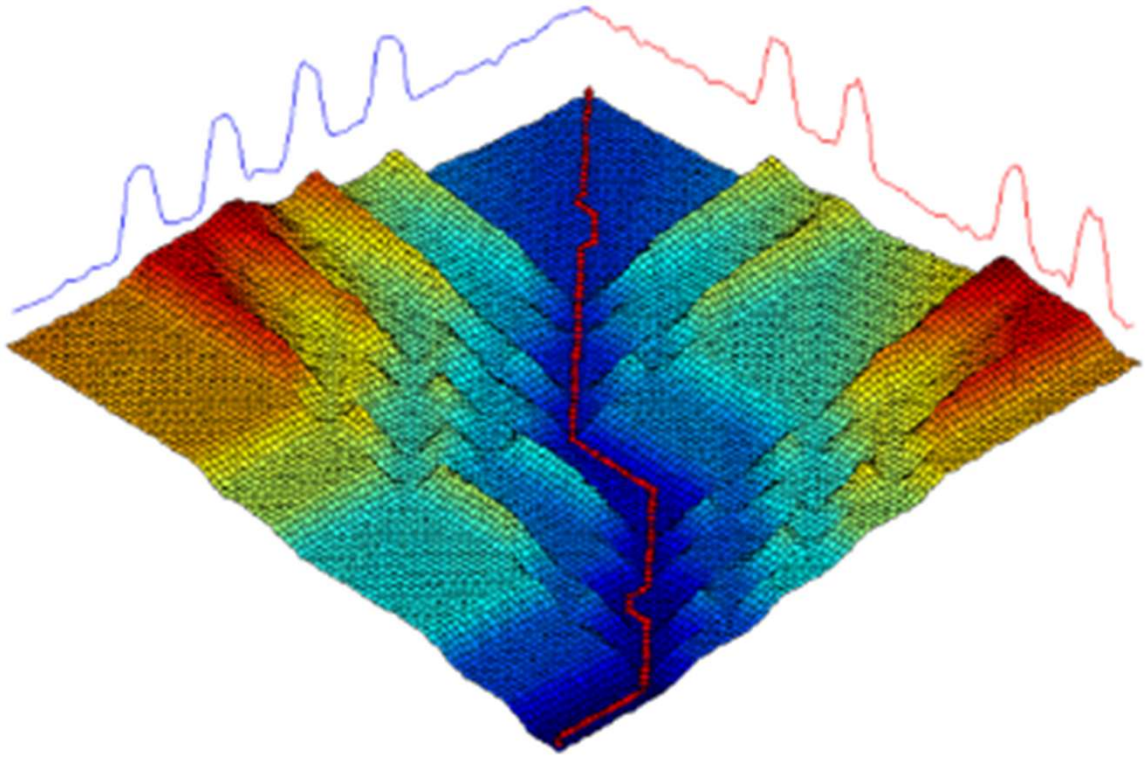
Dynamic Time Warping

Elastic distances align two series while preserving the order of their observations

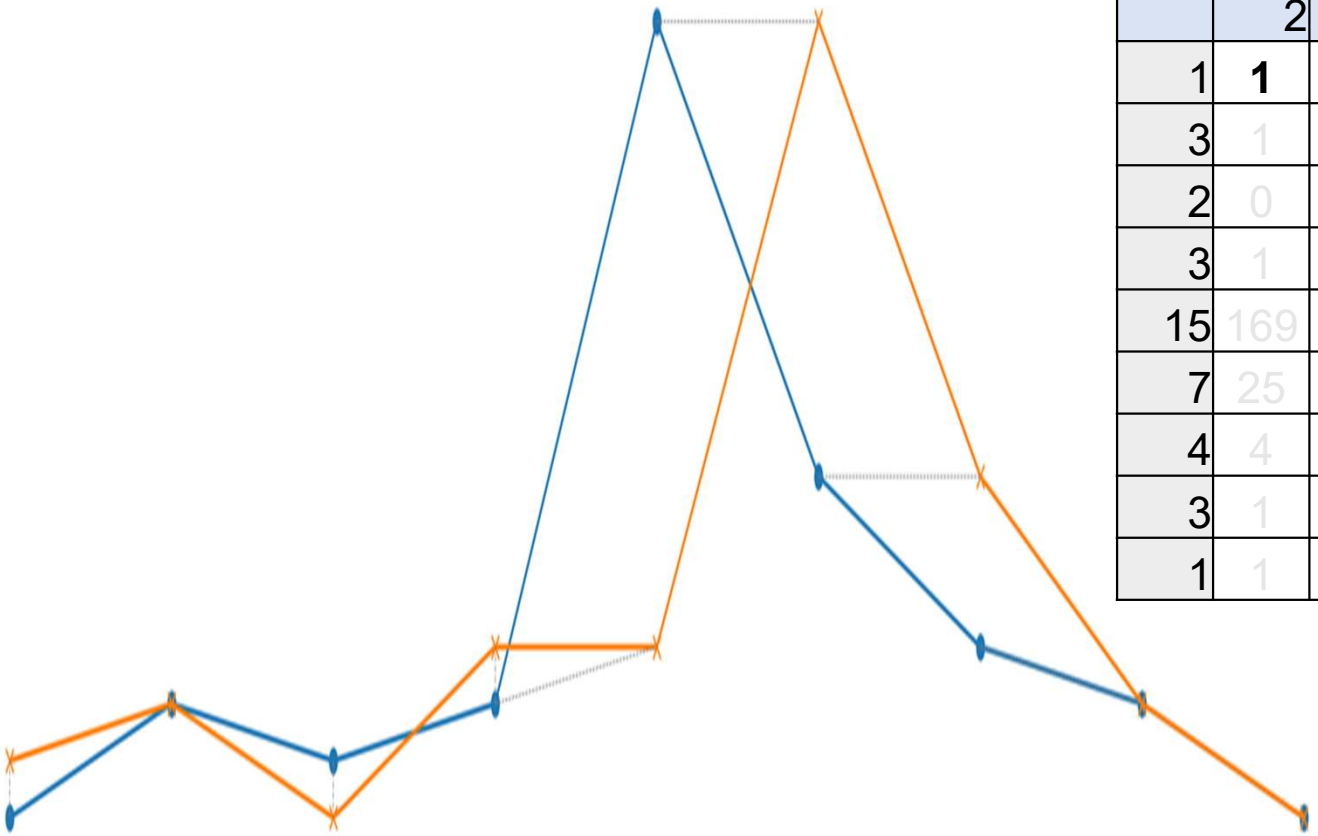
A point may align with more than one point in the other series

Add the local alignment costs along a path

Choose the valid path with the lowest total cost: **optimisation problem**



Dynamic Time Warping (DTW)



	2	3	1	4	4	15	7	3	1
1	**1**	4	0	9	9	196	36	4	0
3	1	**0**	4	1	1	144	16	0	4
2	0	1	**1**	4	4	169	25	1	1
3	1	0	4	**1**	**1**	144	16	0	4
15	169	144	196	121	121	**0**	64	144	196
7	25	16	36	9	9	64	**0**	16	36
4	4	1	9	0	0	121	9	**1**	9
3	1	0	4	1	1	144	16	**0**	4
1	1	4	0	9	9	196	36	4	**0**

Dynamic programming formulation

DTW finds the path through the pointwise distance matrix that minimizes the total distance

---

1: Let C be an $(m+1) \times (m+1)$ matrix initialised to zero, indexed from zero.
2: **for** $i \leftarrow 1$ to m **do**
3: **for** $j \leftarrow 1$ to m **do**
4: $C_{i,j} \leftarrow M_{i,j} + \min(C_{i-1,j-1}, C_{i-1,j}, C_{i,j-1})$
return $C_{m,m}$

See lab sheet 1 optional exercises, implementation is in the lab repository and in aeon

M=

	2	3	1	4	4
1	1	4	0	9	9
3	1	0	4	1	1
2	0	1	1	4	4
3	1	0	4	1	1
15	121	144	196	121	169

C=

	2	3	1	4	4
1	**1**	5	5	14	23
3	2	**1**	5	6	7
2	2	2	**2**	6	10
3	3	2	6	**3**	**4**
15	172	146	198	124	124

Nearest Neighbour with DTW

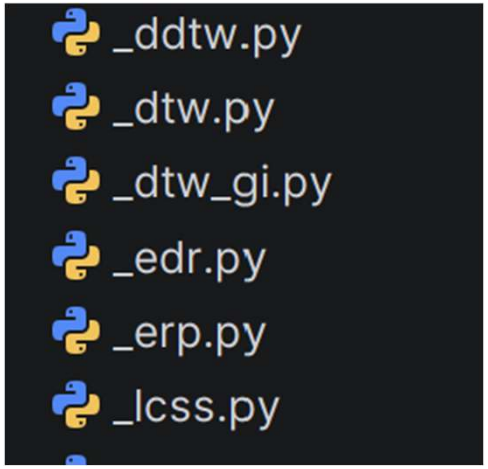
```
from aeon.classification.distance_based import KNeighborsTimeSeriesClassifier

knn = KNeighborsTimeSeriesClassifier(n_neighbors=1, distance="dtw")

knn.fit(X_train, y_train)
pred = knn.predict(X_test)
print(accuracy_score(y_test, pred))
```

aeon classifiers can take 2D or 3D collections directly.

There are a large number of different elastic distance functions available in aeon





Preprocessing is important: from time series to cases

- Imagine a single recording of a person performing several labelled activities
- We could use windowing or a segmenter to make cases
- A segmenter could give us unequal length series
- There may be cross-over periods where tasks are mixed (e.g. transition from standing to running)
- We may want to rescale to look for shape only
- All of these decisions are driven by the application and the desired use case

Evaluation is dependent on the ultimate use case: avoid leakage

Prediction task	A suitable separation
Another independent recording	Separate cases
A new person or device	Hold out people or devices
A later recording session	Hold out sessions
Future observations	Train on the past, test on later values

Conclusions

- A collection of time series contains cases, channels and ordered timepoints
- There are generic ML tasks (classification/regression/clustering) and bespoke time series tasks (segmentation/forecasting)
- Distance functions are primitives that can be used for all tasks to compare whole series.
- Elastic distances such as DTW allow for small misalignments in series when measuring similarity
- Next: construct features for familiar classifiers and ensembles